

步骤一：静态确认

核心任务：确认不是“降智”导致的低级错误。

- [] **重读题面与样例：**人脑手动模拟一遍原题样例。绝对确认自己没有**读错题**（如：漏看正负数、看反输入顺序、下标从0/1开始）。
- [] **扫描常见低级 Bug：**
 - 数组开得够不够大？（线段树 4 倍，双向图边开 2 倍）。
 - 是否涉及多组数据？（检查循环内部清空是否彻底，不能盲目 `memset` 全局）。
 - 有没有忘记开 `long long`？（特别是 `1LL * a * b` 这种乘法中间溢出）。

步骤二：两刀切·缩减范围

核心任务：不要满篇找 Bug，用“两刀切”把问题锁定在某个特定模块。

- **第一刀：切输入与数据结构**
 - 在建图（邻接表）或数据结构（如线段树 `build`）刚结束时，直接打印结构。
 - **判断：**如果初始状态、边数、权值就已经错了，说明问题在**读入或建树**，后面不用看了。
- **第二刀：切核心算法阶段**
 - 如果是 DP，打印前 3 阶段的 `dp` 数组值；如果是图论，打印前 3 次松弛操作。
 - **判断：**对比草稿纸，如果前几步是对的，说明**状态转移/核心逻辑**大体没错，问题在**边界或后半段**。

步骤三：动态追踪

核心任务：利用 `cout` 打印“时间线”，让程序自己暴露轨迹。

- [] **构造极小数据：**大样例无法追踪，立刻手动写一组 $N \leq 5$ 的极小错误样例。
- **安插断点：**在核心循环、DFS/BFS 内部，打印当前处理的节点、状态和关键变量（如 `u`, `v`, `dist`, `dp[i]`）。

- [] **对齐时空线**：运行程序，对照草稿纸上人类应有的逻辑。找出程序轨迹与人类逻辑发生“分叉”的那一行。